

Introdução a Grafos — Parte I

Universidade Estadual do Sudoeste da Bahia - UESB

2026.1

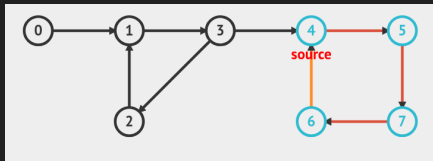
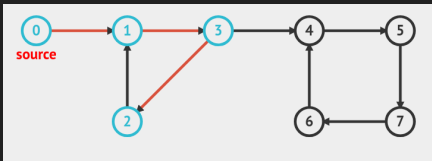
O que é um Grafo?

Um grafo é um par de conjuntos (V, E) , onde:

- ▶ V é um conjunto de elementos chamados **vértices**.
- ▶ E é um conjunto de pares $\{a, b\}$ chamados **arestas**, onde $a, b \in V$.
- ▶ Em um grafo **não direcionado**: $\{a, b\} = \{b, a\}$.
- ▶ Em um grafo **direcionado**: $\{a, b\} \neq \{b, a\}$.

Definições Importantes

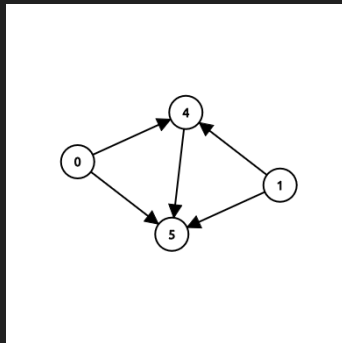
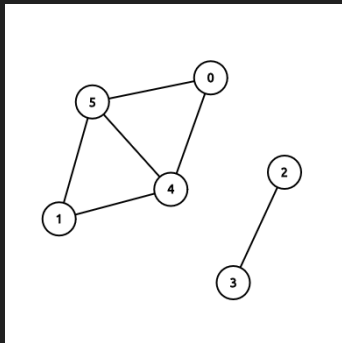
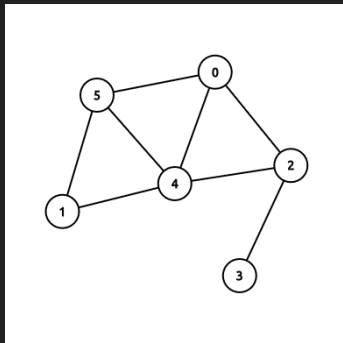
- ▶ **Caminho:** sequência de vértices adjacentes inicializada e finalizada em 2 vértices distintos. Os vértices não se repetem.
- ▶ **Ciclo:** um caminho acrescido de uma aresta que liga o vértice inicial ao final.



Obs.: imagens geradas por <https://visualgo.net/en/dfsbfbs>

Definições Importantes

- ▶ **Grafo conexo:** grafo em que, para todo par de vértices $\{a, b\} \in V$, existe um caminho ligando-os.
- ▶ **Componente conexo:** um subgrafo conexo maximal de um grafo.



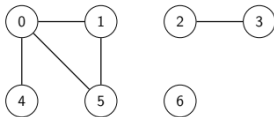
Obs.: imagens geradas por https://csacademy.com/app/graph_editor/

Representação de Grafos — Lista de Adjacência

Há diversas formas de representar um grafo em código. A mais utilizada é a **lista de adjacência**.

- ▶ Um vetor de vetores, onde a posição i armazena todos os vértices vizinhos de i .

```
v[0] = {1,4,5}
v[1] = {0,5}
v[2] = {3}
v[3] = {2}
v[4] = {0}
v[5] = {0,1}
v[6] = {}
```



Complexidade de espaço: $O(|V| + |E|)$

Representação de Grafos — Leitura

A maioria das entradas de grafos fornece uma lista de arestas.

Código de leitura:

Exemplo de entrada:

```
5 5
1 2
1 3
1 4
2 3
5 4
```

```
int n, m;
cin >> n >> m;
for (int i = 0; i < m; i++) {
    int a, b;
    cin >> a >> b;
    a--, b--;
    adj[a].push_back(b);
    adj[b].push_back(a);
}
```

Representação de Grafos — Outras Formas

- ▶ **Matriz de adjacência:** a matriz `adj[n][n]` indica com 1 ou 0 se há uma aresta ligando os vértices i e j na posição (i, j) .
Complexidade de espaço: $O(|V|^2)$
- ▶ **Vetor de arestas:** o vetor `vector<pair<int,int>> E(m)` armazena todas as arestas do grafo.
Complexidade de espaço: $O(|E|)$

Problema 1 — Building Roads

Enunciado: Byteland possui n cidades e m estradas entre elas. O objetivo é construir novas estradas de forma que haja um caminho entre quaisquer duas cidades. Encontre o **número mínimo** de estradas a serem construídas e indique quais são elas.

Entrada:

```
4 2
1 2
3 4
```

Saída:

```
1
2 3
```

Restrições: $1 \leq n \leq 10^5$, $1 \leq m \leq 2 \cdot 10^5$

Percorrendo um Grafo — DFS e BFS

Há 2 formas principais de se percorrer um grafo:

- ▶ **DFS** (*Depth-first search*): o algoritmo percorre o grafo de forma recursiva, onde cada vizinho do nó de origem se torna a nova origem da busca.
- ▶ **BFS** (*Breadth-first search*): o algoritmo percorre o grafo a partir de uma fila, onde cada vizinho do nó de origem é analisado antes de continuar a busca em profundidade.

Visualização Gráfica

É possível visualizar ambas as buscas de forma interativa no site:

<https://visualgo.net/en/dfsbfbs>

DFS — Implementação

```
vector<vector<int>> adj(N);  
vector<bool> visited(N);  
  
void dfs(int s) {  
    if (visited[s]) return;  
    visited[s] = true;  
  
    // processa o nó s  
    for (auto u : adj[s]) {  
        dfs(u);  
    }  
}
```

- ▶ Complexidade temporal: $O(|V| + |E|)$
- ▶ Complexidade espacial: $O(|V|)$
- ▶ Aplicações: conectividade, grafo bipartido, ciclos, ordenação topológica, etc.

Conectividade com DFS

É possível analisar quantos **componentes conexos** há em um grafo não direcionado a partir da DFS. Como a DFS aplicada no vértice u visita todos os vértices que possuem caminho até u , todos os vértices visitados em uma chamada formam um componente conexo.

```
int countComponents = 0;
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        countComponents++;
        dfs(i);
    }
}
```

Building Roads

Implemente a solução utilizando DFS e conectividade.

Problema 2 — Building Teams

Enunciado: Há n alunos na turma de Uolevi, e m amizades entre eles. Divida os alunos em **dois times** de forma que nenhum par de amigos esteja no mesmo time. Se não for possível, imprima IMPOSSIBLE.

Entrada:

```
5 3
1 2
1 3
4 5
```

Saída:

```
1 2 2 1 2
```

Restrições: $1 \leq n \leq 10^5$, $1 \leq m \leq 2 \cdot 10^5$

Problema da Coloração de Mapas

É possível colorir um mapa com k cores de forma que territórios vizinhos nunca tenham a mesma cor?

Um mapa pode ser tratado como um grafo, onde os **vértices** são os territórios e uma **aresta** entre a e b indica que são vizinhos.

- ▶ **Teorema das 4 cores:** não são necessárias mais que 4 cores para colorir qualquer mapa nessas condições.
- ▶ **3-coloring problem:** para $k = 3$, o problema é NP-Completo.
- ▶ **2-coloring problem:** verificável com DFS!

Grafo Bipartido — 2-Coloring Problem

Em um **grafo bipartido**, é possível dividir os vértices em 2 grupos de forma que vértices do mesmo grupo não sejam vizinhos entre si.

Visualize o algoritmo em <https://visualgo.net/en/dfsdfs>

Hora de Implementar

Building Teams

Implemente a verificação de grafo bipartido com DFS.

Building Teams — Solução

- ▶ Cria-se um vetor `color` de tamanho n , onde a posição i representa a cor do vértice i .
- ▶ Se `color[i] == 0`, o vértice ainda não foi visitado.
- ▶ Se algum vértice possuir a mesma cor que um de seus vizinhos, então não é possível colorir o grafo — imprima IMPOSSIBLE.

Building Teams — Código da DFS

```
vector<vector<int>> adj(N);  
vector<int> color(N);  
bool possible = true;  
  
void dfs(int i, int c) {  
    color[i] = c;  
    int newColor = (c == 1 ? 2 : 1);  
  
    for (auto v : adj[i]) {  
        if (color[v] == 0)  
            dfs(v, newColor);  
        else if (color[v] == newColor)  
            continue;  
        else {  
            possible = false;  
            return;  
        }  
    }  
}
```

Problema 3 — Message Route

Enunciado: A rede de Syrjälä possui n computadores e m conexões. Uolevi está no computador 1 e Maija no computador n . Determine se é possível enviar uma mensagem de Uolevi para Maija e, se sim, qual é o **menor caminho** (em número de computadores) entre eles.

Entrada:

```
5 5
1 2
1 3
1 4
2 3
5 4
```

Saída:

```
3
1 4 5
```

Restrições: $2 \leq n \leq 10^5$, $1 \leq m \leq 2 \cdot 10^5$

Fila (queue)

Para o próximo algoritmo de busca (BFS), é necessário percorrer o grafo utilizando uma **fila**. Uma fila é uma estrutura *First In, First Out* (FIFO), com 3 operações principais:

- ▶ **push**: insere o elemento no final da fila.
- ▶ **pop**: remove o primeiro elemento da fila.
- ▶ **front**: consulta o primeiro elemento da fila.

```
queue<int> q;  
q.push(1);           // [1]  
q.push(3);           // [1, 3]  
q.push(4);           // [1, 3, 4]  
q.pop();             // [3, 4]  
cout << q.front() << endl; // 3
```

BFS — Funcionamento

- ▶ A fila serve como a lista de nós descobertos mas ainda não processados.
- ▶ Para cada vizinho não visitado de um nó, insere-se esse vizinho na fila.
- ▶ Quando todos os vizinhos de um nó forem processados, continua-se a busca pelo próximo elemento da fila.
- ▶ Complexidade: $O(|V| + |E|)$
- ▶ Aplicações: menor distância, Dijkstra, etc.

BFS — Implementação

```
vector<vector<int>> adj(N);  
vector<bool> visited(N);  
  
void bfs(int x) {  
    queue<int> q;  
    visited[x] = true;  
    q.push(x);  
    while (!q.empty()) {  
        int s = q.front(); q.pop();  
        // processa o nó s  
        for (auto u : adj[s]) {  
            if (visited[u]) continue;  
            visited[u] = true;  
            q.push(u);  
        }  
    }  
}
```

Menor Caminho com BFS

Com a BFS, é possível encontrar o **menor caminho** entre 2 vértices em um grafo não direcionado e não valorado. A fila garante que os vértices são processados em ordem crescente de distância à origem.

Para encontrar o menor caminho entre A e B :

- 1º Criar um vetor de distância, onde $\text{dist}[i]$ representa a distância de i até a origem:

```
vector<int> dist(N, -1);
```

- 2º Criar um vetor de pai, onde $\text{pai}[i]$ representa o vértice anterior de i no caminho de A até B :

```
vector<int> pai(N, -1);
```


Menor Caminho com BFS — Implementação

```
void bfs(int x) {  
    queue<int> q;  
    visited[x] = true;  
    q.push(x);  
    pai[x] = x; dist[x] = 0;  
  
    while (!q.empty()) {  
        int s = q.front(); q.pop();  
        for (auto u : adj[s]) {  
            if (visited[u]) continue;  
            visited[u] = true;  
            q.push(u);  
            pai[u] = s;  
            dist[u] = dist[s] + 1;  
        }  
    }  
}
```

Hora de Implementar

Message Route

Implemente a solução utilizando BFS e o vetor de pais.

Message Route — Solução

Após aplicar a BFS e construir o vetor de pais, recupera-se o caminho de A até B com a seguinte função:

```
vector<int> ret;  
while (pai[a] != -1) {  
    ret.push_back(a);  
    if (pai[a] == a)  
        break;  
    else  
        a = pai[a];  
}  
reverse(ret.begin(), ret.end());
```

Accepted

Tempo: 0.00s — Memória: 0 KB

Nos vemos na próxima aula.